



ساختمان داده‌ها و الگوریتم‌ها

کوییز دوم - ساختمان‌های داده

شماره دانشجویی:

نام و نام خانوادگی:

تاریخ: ۱۴۰۴/۰۸/۲۵

۴۰ نمره

۱. جنگ انجمنی

به یک عضو در آرایه n عضوی غالب می‌گوییم اگر تعداد تکرار آن در آرایه اکیدا بیشتر از $\lfloor \frac{n}{3} \rfloor$ باشد. به شما یک آرایه n عضوی داده می‌شود، الگوریتمی بر پایه stack از $O(n)$ پیدا کنید که عضو غالب آن را پیدا کند یا بگوید چنین عضوی وجود ندارد.

پاسخ:

ایده اصلی این الگوریتم، «حذف جفتی» است. از آنجایی که عضو غالب بیش از نیمی از آرایه را تشکیل می‌دهد، اگر هر بار یک عضو غالب را با یک عضو غیرغالب حذف کنیم، در انتها باز هم عضوی از اعضای غالب باقی خواهد ماند. پشته به ما کمک می‌کند این فرآیند «حذف» را شبیه‌سازی کنیم.

الگوریتم: الگوریتم در دو مرحله انجام می‌شود:

مرحله ۱: پیدا کردن کاندیدای احتمالی

۱. یک پشته (stack) خالی ایجاد کنید.

۲. آرایه a را از ابتدا تا انتها پیمایش کنید (از $i = 0$ تا $n - 1$):

- اگر پشته خالی بود، عنصر فعلی ($a[i]$) را به پشته اضافه کنید (push).
- اگر پشته خالی نبود و $a[i]$ برابر با عنصر بالای پشته ($\text{stack.top}()$) بود، $a[i]$ را به پشته اضافه کنید. (این معادل افزایش شمارنده کاندیدا است).
- اگر پشته خالی نبود و $a[i]$ مخالف عنصر بالای پشته بود، یک عنصر از پشته حذف کنید (pop). (این معادل «حذف جفتی» یا کاهش شمارنده کاندیدا است).

۳. پس از پایان پیمایش آرایه:

- اگر پشته خالی باشد، هیچ عضو غالبی وجود ندارد. الگوریتم پایان می‌یابد.
- اگر پشته خالی نباشد، عنصر بالای پشته ($\text{stack.top}()$) تنها کاندیدای ممکن برای عضو غالب است.

مرحله ۲: تأیید کاندیدا عنصر باقی‌مانده در پشته در مرحله ۱، فقط یک «کاندیدا» است و لزوماً عضو غالب نیست (مثلاً در آرایه $[A, B, C]$ ، عنصر C باقی می‌ماند اما غالب نیست). باید در یک پیمایش دوم، فراوانی این کاندیدا را بشماریم.

۱. کاندیدا را ذخیره کنید: $\text{candidate} = \text{stack.top}()$.

۲. یک شمارنده $\text{count} = 0$ ایجاد کنید.

۳. دوباره کل آرایه a را پیمایش کنید:

- اگر $a[i] == \text{candidate}$ بود، count را یکی افزایش دهید.
- پس از پایان پیمایش، بررسی کنید:
- اگر $\lfloor \frac{n}{4} \rfloor > \text{count}$ ، آنگاه candidate عضو غالب است و آن را برگردانید.
- در غیر این صورت، عضو غالبی وجود ندارد.

تحلیل زمانی: مرحله ۱ یک بار آرایه را پیمایش می‌کند. هر عنصر حداکثر یک بار push و یک بار pop می‌شود. پس این مرحله $O(n)$ است. مرحله ۲ نیز یک بار آرایه را پیمایش می‌کند و $O(n)$ است. بنابراین، کل الگوریتم از مرتبه زمانی $O(n)$ است.

۲. مینیچکا

۶۰ نمره

یک آرایه n عضوی به شما داده شده است. یک عدد k نیز به شما داده می‌شود که از اندازه آرایه بزرگ‌تر نیست. مینیمم همه بازه‌های به طول k در آرایه را چاپ کنید.
به عبارت دیگر شما باید $n - k + 1$ عدد چاپ کنید که i امین آن‌ها برابر مینیمم بازه $a_i, a_{i+1}, \dots, a_{i+k-1}$ است. الگوریتم شما باید از اردر زمانی $O(n)$ باشد.

پاسخ:

برای حل این مسئله در زمان $O(n)$ ، می‌توانیم از یک ساختار داده به نام «صف دوطرفه» (Deque) استفاده کنیم. این صف ایندکس‌های عناصر آرایه را ذخیره می‌کند. ما صف را طوری مدیریت می‌کنیم که همواره ایندکس‌های عناصری که در پنجره فعلی قرار دارند و پتانسیل مینیمم بودن را دارند، در آن نگهداری شوند.
قوانین نگهداری صف به این صورت است:

- صف همواره به صورت صعودی بر اساس *مقدار* عناصر آرایه (که ایندکسشان در صف است) مرتب می‌ماند.
- صف فقط شامل ایندکس‌هایی از پنجره فعلی است.

الگوریتم:

۱. یک صف دوطرفه خالی (مثلاً dq) ایجاد کنید.
۲. پردازش پنجره اول (از ۰ تا $k-1$): برای هر i از ۰ تا $k-1$:
 - تا زمانی که dq خالی نیست و عنصر انتهایی صف ($a[dq.back()]$) بزرگتر یا مساوی $a[i]$ است، انتهای صف را حذف کنید. (زیرا $a[i]$ از آن کوچک‌تر است و دیرتر از پنجره خارج می‌شود، پس عنصر انتهایی صف دیگر هرگز مینیمم نخواهد بود).
 - ایندکس i را به انتهای dq اضافه کنید.
۳. پردازش بقیه آرایه (از k تا $n-1$): برای هر i از k تا $n-1$:
 - چاپ مینیمم قبلی: مینیمم پنجره قبلی (که در $i-1$ تمام می‌شد) برابر $a[dq.front()]$ است. آن را چاپ کنید.
 - حذف عناصر خارج از پنجره: تا زمانی که dq خالی نیست و ایندکس ابتدای صف ($dq.front()$) کوچکتر یا مساوی $i-k$ است، ابتدای صف را حذف کنید. (این ایندکس دیگر در پنجره فعلی $[i-k+1, i]$ قرار ندارد).
 - حذف عناصر بزرگتر: تا زمانی که dq خالی نیست و عنصر انتهایی صف ($a[dq.back()]$) بزرگتر یا مساوی $a[i]$ است، انتهای صف را حذف کنید. (مانند مرحله ۲).
 - ایندکس i را به انتهای dq اضافه کنید.
۴. چاپ آخرین مینیمم: پس از پایان حلقه، مینیمم آخرین پنجره $a[dq.front()]$ است. آن را نیز چاپ کنید.

(توجه: پیاده‌سازی باید $n - k + 1$ عدد چاپ کند. در روش بالا، ما مینیمم پنجره قبلی را $*$ قبل $*$ از پردازش عنصر i چاپ می‌کنیم و مینیمم آخرین پنجره را پس از حلقه چاپ می‌کنیم.)

تحلیل زمانی: هر ایندکس دقیقاً یک بار به صف اضافه شده و حداکثر یک بار از آن حذف می‌شود (چه از ابتدا و چه از انتها). بنابراین، تمام عملیات‌های روی صف در مجموع $O(n)$ طول می‌کشد و الگوریتم از مرتبه زمانی $O(n)$ است.